Práctica 5: Problemas de optimización en redes con Pyomo.

1. Introducción

En esta práctica se mostrará como resolver problemas de optimización en redes utilizando Pyomo. Se trabajará con ejemplos ya que es más sencilla la explicación. Concretamente, se trabajaran los siguientes aspectos:

- Modelizar y resolver con Pyomo problemas de optimización en redes: problemas del transporte, problemas de asignación, problemas de flujo máximo, ...
- Resolver con Pyomo problemas de flujo en redes a coste mínimo.
- Relación entre problemas de optimización en redes y problemas de flujo en redes a coste mínimo.

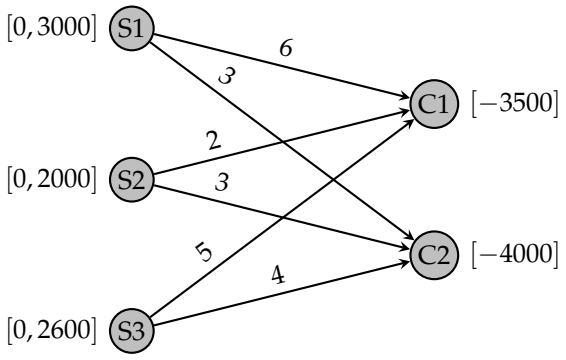
2. Ejemplo de problema del transporte

Vamos a resolver el problema de Netflix de las transparencias del Tema 3 teniendo en cuenta los siguientes datos:

- Hay 3 servidores con capacidades de 3000, 2000 y 2600 MB, respectivamente,
- hay 2 clientes con demandas de 3500 y 4000 MB, respectivamente,
- Las tasas de transmisión (en milisegundos por MB) entre servidores y clientes vienen determinadas por la siguiente tabla:

	C_1	C_2
S_1	6	3
S_2	2	3
S_3	5	4

Este problema se puede representar con la siguiente red:



La formulación matemática como un problema de programación lineal sería:

$$
\begin{array}{lrcl}
\textbf{minimizar} & 6f_{11}+3f_{12}+2f_{21}+3f_{22}+5f_{31}+4f_{32} & & \\
\textbf{sujeto a} & f_{11}+f_{12} & \leq & 3000 \\
& f_{21}+f_{22} & \leq & 2000 \\
& f_{31}+f_{32} & \leq & 3600 \\
& f_{11}+f_{21}+f_{31} & = & 3500 \\
& f_{12}+f_{22}+f_{32} & = & 4000 \\
& \boldsymbol{f} & \geq & 0
\end{array}
$$

siendo f_{ij} la variable que proporciona el flujo que pasaría desde el servidor i al cliente j.

Utilizaremos el modelo abstracto con el objetivo de poder reutilizar el código para posteriores ejercicios. El archivo a utilizar será `Transporte.py`[1] y su contenido será:

```
import pyomo.environ as pyo

# Crear el modelo
model = pyo.AbstractModel()

# Definir los Parámetros
model.Origenes = pyo.Set() # Conjunto de Origenes
model.Destinos = pyo.Set() # Conjunto de Destinos

model.Oferta = pyo.Param(model.Origenes)  # Oferta de los origenes (se podría exigir que fuesen enteros para garantizar la unimodularidad)
model.Demanda = pyo.Param(model.Destinos)  # Demanda de los destinos (se podría exigir que fuesen enteros para garantizar la unimodularidad)
model.Costes = pyo.Param(model.Origenes,model.Destinos) # Coste unitario Origen-Destino


# Definir las variables y su dominio
model.Flujo = pyo.Var(model.Origenes,model.Destinos, domain=pyo.NonNegativeReals) # Por la propiedad de unimodularidad NO sería necesario exigir que sean valores enteros si las ofertas y las demandas son también números enteros

# Establecer la función objetivo en dos pasos:
# 1. se define la expresión de la función objetivo
def obj_expression(m):
return pyo.summation(m.Costes, m.Flujo)
# 2. Se establece la función objetivo
model.Coste = pyo.Objective(rule=obj_expression,sense=pyo.minimize)

# Establecer la restricción de Oferta:
# 1. se define la expresión del bloque de restricciones
def oferta_constraint_rule(m,i):
# devuelve la expresión para el origen i
return sum(m.Flujo[i,j] for j in m.Destinos) <= m.Oferta[i]

# 2. Se crea la restricción de oferta máxima
model.maxOferta = pyo.Constraint(model.Origenes,rule=oferta_constraint_rule)

# Establecer la restricción de demanda:
# 1. se define la expresión del bloque de restricciones
def demanda_constraint_rule(m,j):
# devuelve la expresión para el Destino j
return sum(m.Flujo[i,j] for i in m.Origenes) == m.Demanda[j]

# 2. Se crea la restricción de demanda mínima
model.minDemanda = pyo.Constraint(model.Destinos,rule=demanda_constraint_rule)

###############################################################################
# Para que se muestren directamente al ejecutar desde python (con  p ej. IDLE)
solver = pyo.SolverFactory('glpk')
# Se crea una instancia: modelo abstracto + datos
instancia = model.create_instance('Netflix_1.dat')

# Muestra la instancia sin resolver para comprobar si lo ha hecho bien:
instancia.pprint()

#Se resuelve la instancia
results = solver.solve(instancia)

# Imprime la información sobre la ejecución del solver
print(results)
```

[1] se puede descargar de la página web de la asignatura

```
57
58
59 # Salida formateada:
60 print("\nSolución:")
61 print(f"\nCoste Total = {instancia.Coste()}")
62
63 print("\nVariables (!=0) en el óptimo:")
64 for v in instancia.component_data_objects(pyo.Var, active=True):
65 # Sólo se imprimen los datos distintos de 0
66     if v() > 0:
67         print(f"\t{v} = {v()}")
68
69 print("\nRestricciones en el óptimo:")
70 for c in instancia.component_data_objects(pyo.Constraint, active=True):
71     print(f"\t{c} = {c()}")
72
73 # Muestra la instancia resuelta
74 instancia.display()
```

El fichero con los datos del problema sería `Netflix_1.dat`,[2] cuyo contenido sería:

```
1  ## Ejemplo Netflix - Data en formato AMPL
2
3  set Origenes:= S1 S2 S3;
4  set Destinos:= C1 C2;
5
6  param Oferta:=
7    S1 3000
8    S2 2000
9    S3 2600;
10
11 param Demanda:=
12   C1 3500
13   C2 4000;
14
15 param Costes: C1 C2 :=
16   S1  6  3
17   S2  2  3
18   S3  5  4;
```

Para obtener la solución se ejecutaría el siguiente código en el *prompt* de Anaconda:

```
1 python Transporte.py
```

El resultado se mostrará por pantalla.

Como se ha resuelto el problema como un problema de programación lineal se puede realizar un análisis de sensibilidad como el visto en una práctica anterior. Así se podrían responder preguntas del tipo:

- ¿Qué ocurre si el Cliente 2 demanda 10MB más? ¿Cuál sería ahora el tiempo total de transmisión? (Sol: aumenta en 40 msg ¿por qué?)
- ¿Qué ocurre si el Servidor 2 aumenta su capacidad en 10MB? ¿Cuál sería ahora el tiempo total de transmisión? (Sol: disminuye en 30 msg ¿por qué?)
- ¿Y si el Servidor 2 aumenta su capacidad en 1600MB? (Sol: resolver otra vez el problema ¿por qué?)
- ¿Como debería de ser modificada la tasa de envío de datos del servidor 2 al Cliente2 para que sea beneficioso enviar datos a través de esa línea? (Sol: disminuir en al menos 2msg)

2.1. Arcos ausentes

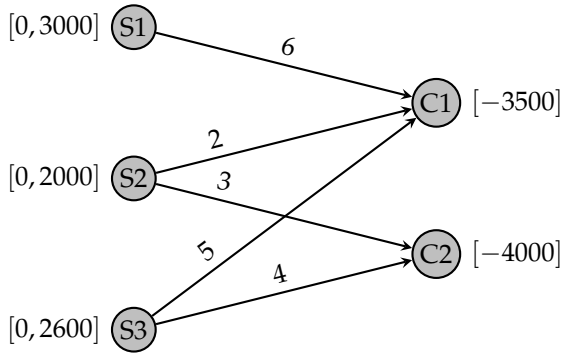
Hay situaciones en las que alguna conexión entre un origen y un destino no es posible (no existe carretera, no hay línea de fibra, ...). En estas situaciones lo que se suele hacer es asignarle un coste muy alto al arco

[2] Disponible en la web de la asignatura

asociado a esa conexión y de esta manera el propio algoritmo no seleccionará ese arco si estamos minimizando. Sin embargo, Pyomo tiene otras herramientas que me permiten trabajar directamente con los arcos como índices y así podríamos definir como parámetro el conjunto de arcos con esta instrucción:

```
model.Arcos = pyo.Set(within=model.Origenes*model.Destinos)
```

De esta manera simplemente habría que considerar el conjunto de arcos donde el arco en cuestión esté ausente. Por ejemplo, si en el problema anterior de Netflix, la conexión entre el Servidor 1 y el Cliente 2 no fuese posible la red sería la siguiente:



Entonces el código de Python habría que modificarlo de la siguiente manera:

```
# Modelo abstracto para el problema del transporte genérico
# Con este código, a las conexiones que no existan no se ponen

import pyomo.environ as pyo

# Crear el modelo
model = pyo.AbstractModel()

# Definir los Parámetros
model.Origenes = pyo.Set() # Conjunto de Origenes
model.Destinos = pyo.Set() # Conjunto de Destinos

model.Arcos = pyo.Set(within=model.Origenes*model.Destinos) # Sólo arcos disponibles

model.Oferta = pyo.Param(model.Origenes)  # Oferta de los origenes (se podría exigir que
    fuesen enteros para garantizar la unimodularidad)
model.Demanda = pyo.Param(model.Destinos)  # Demanda de los destinos (se podría exigir que
    fuesen enteros para garantizar la unimodularidad)
model.Costes = pyo.Param(model.Arcos) # Coste unitario Arcos


# Definir las variables y su dominio
model.Flujo = pyo.Var(model.Arcos, domain=pyo.NonNegativeReals) # Por la propiedad de
    unimodularidad NO sería necesario exigir que sean valores enteros si las ofertas y las
    demandas son también números enteros

# Establecer la función objetivo en dos pasos:
# 1. se define la expresión de la función objetivo
def obj_expression(m):
return pyo.summation(m.Costes, m.Flujo)
# 2. Se establece la función objetivo
model.Coste = pyo.Objective(rule=obj_expression,sense=pyo.minimize)

# Establecer la restricción de Oferta:
# 1. se define la expresión del bloque de restricciones
def oferta_constraint_rule(m,i):
# devuelve la expresión para el origen i
return sum(m.Flujo[k,j] for (k,j) in m.Arcos if k == i) <= m.Oferta[i]

```

```python
# 2. Se crea la restricción de oferta máxima
model.maxOferta = pyo.Constraint(model.Origenes,rule=oferta_constraint_rule)

# Establecer la restricción de demanda:
# 1. se define la expresión del bloque de restricciones
def demanda_constraint_rule(m,j):
# devuelve la expresión para el Destino j
return sum(m.Flujo[i,k] for (i,k) in m.Arcos if k == j) == m.Demanda[j]

# 2. Se crea la restricción de demanda mínima
model.minDemanda = pyo.Constraint(model.Destinos,rule=demanda_constraint_rule)

#############################################################################
# Para que se muestren directamente al ejecutar desde python (con  p ej. IDLE)
solver = pyo.SolverFactory('glpk')
# Se crea una instancia: modelo abstracto + datos
instancia = model.create_instance('Netflix_2.dat')

# Muestra la instancia sin resolver para comprobar si lo ha hecho bien:
instancia.pprint()

#Se resuelve la instancia
results = solver.solve(instancia)

# Imprime la información sobre la ejecución del solver
print(results)

# Salida formateada:
print("\nSolución:")
print(f"\nCoste Total = {instancia.Coste()}")

# OJO!!: Para el problema abstracto se trabaja con la instancia y se utiliza
    component_data_objects
# para acceder a las variables del problema.
print("\nVariables (!=0) en el óptimo:")
for v in instancia.component_data_objects(pyo.Var, active=True):
# Sólo se imprimen los datos distintos de 0
    if v() > 0:
        print(f"\t{v} = {v()}")

print("\nRestricciones en el óptimo:")
for c in instancia.component_data_objects(pyo.Constraint, active=True):
    print(f"\t{c} = {c()}")

# Muestra la instancia resuelta
instancia.display()
```

El fichero con los datos del problema sería `Netflix_2.dat`,[3] cuyo contenido sería:

```
## Ejemplo Netflix con arcos - Data en formato AMPL

set Origenes:= S1 S2 S3;
set Destinos:= C1 C2;

set Arcos:= (S1,C1) (S1,C2) (S2,C1) (S2,C2) (S3,C1) (S3,C2);

param Oferta:=
S1 3000
S2 2000
S3 2600;

param Demanda:=
C1 3500
C2 4000;

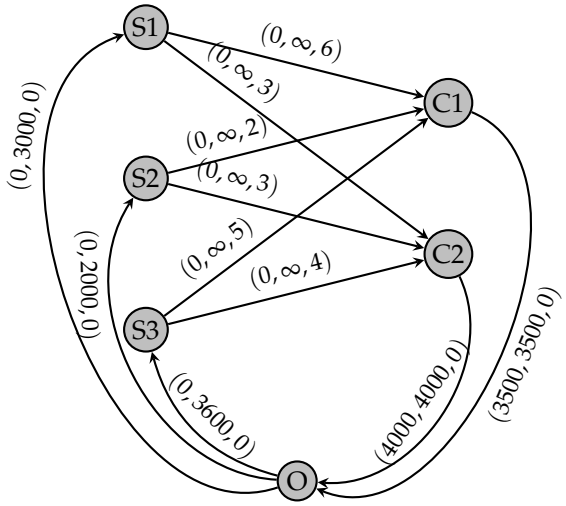
```

[3]Disponible en la web de la asignatura

```
param Costes: C1 C2 :=
S1   6   3
S2   2   3
S3   5   4;
```

3. Problemas de flujo en redes a coste mínimo (PFCM)

Una característica que tienen la mayor parte de los problemas de optimización en redes es que se pueden formular como problemas de flujo en redes a coste mínimo (PFM).[4] Por ejemplo, el problema de Netflix se podría representar como:



Recordar que la formulación genérica de un problema de flujo en redes a coste mínimo sería la siguiente:

$$\begin{aligned} \textbf{minimizar} \quad & \sum_{k \in M} c_k f_k \\ \textbf{sujeto a} \quad & \sum_{k \in M_{O_i}} f_k - \sum_{k \in M_{T_i}} f_k = 0,\ i \in N \\ & l_k \leq f_k \leq u_k,\ k \in M \end{aligned}$$

donde,

- N y M son el conjunto de **arcos y nodos**, respectivamente
- Para $i \in N$,
  - M_{O_i} es el conjunto de arcos que se **originan** en el nodo i,
  - M_{T_i} es el conjunto de arcos que **terminan** en el nodo i
- Para $k \in M$,
  - l_k y u_k denotan la **cantidad mínima y máxima**, respectivamente, que puede pasar por el arco k,
  - c_k es el **coste** asociado a cada arco k,
  - f_k es la **cantidad de flujo** que enviamos por el arco k.

[4] Ver apuntes del Tema 5.

Entonces, un código de Pyomo para representar este tipo de problemas sería:[5]

```
# Modelo para el problema de flujo a coste mínimo (PFCM) sin  flujos externos

import pyomo.environ as pyo

# Crear el modelo
model = pyo.AbstractModel()

# Definir los Parámetros
model.Nodos = pyo.Set() # Conjunto de Nodos

model.Arcos = pyo.Set(within=model.Nodos*model.Nodos) # Sólo arcos disponibles

model.Costes = pyo.Param(model.Arcos, initialize= 0) # Coste unitario Arcos (se inicializa a 0)
model.l = pyo.Param(model.Arcos, initialize= 0) # Capacidad mínima Arcos (se inicializa a 0)
model.u = pyo.Param(model.Arcos, within=pyo.Any, initialize= None) # Capacidad máxima Arcos (se inicializa a None)

# Definir las variables, su dominio y las cotas
# se define la expresión de los pares de (cota_inferior, cota_superior) para las variables
def Cotas(model, i,j):
 return (model.l[i,j], model.u[i,j])

model.Flujo = pyo.Var(model.Arcos, domain=pyo.NonNegativeReals, bounds=Cotas) # Por la propiedad de unimodularidad NO sería necesario exigir que sean valores enteros si las ofertas y las demandas son también números enteros

# Establecer la función objetivo en dos pasos:
# 1. se define la expresión de la función objetivo
def obj_expression(m):
 return pyo.summation(m.Costes, m.Flujo)
# 2. Se establece la función objetivo
model.Coste = pyo.Objective(rule=obj_expression,sense=pyo.minimize)

# Establecer la restricción de Conservación de flujo:
def conservacion_constraint_rule(m,i):
# devuelve la expresión para el nodo i: el flujo que sale es igual al flujo que entra
 return sum(m.Flujo[k,j] for (k,j) in m.Arcos if k == i) - sum(m.Flujo[j,k] for (j,k) in m.Arcos if k == i) == 0
# 2. Se crea la restricción de oferta máxima
model.Conservacion = pyo.Constraint(model.Nodos,rule=conservacion_constraint_rule)


##############################################################################
# Para que se muestren directamente al ejecutar desde python (con  p ej. IDLE)
solver = pyo.SolverFactory('glpk')
# Se crea una instancia: modelo abstracto + datos
instancia = model.create_instance('PFCM_1.dat')

# Muestra la instancia sin resolver para comprobar si lo ha hecho bien:
instancia.pprint()

#Se resuelve la instancia
results = solver.solve(instancia)

# Imprime la información sobre la ejecución del solver
print(results)

# Salida formateada:
print("\nSolución:")
print(f"\nCoste Total = {instancia.Coste()}")

# OJO!!: Para el problema abstracto se trabaja con la instancia y se utiliza
```

[5]Se puede descargar de la web de la asignatura

```
    component_data_objects
# para acceder a las variables del problema.
print("\nVariables (!=0) en el óptimo:")
for v in instancia.component_data_objects(pyo.Var, active=True):
# Sólo se imprimen los datos distintos de 0
 if v() > 0:
      print(f"\t{v} = {v()}")

print("\nRestricciones en el óptimo:")
for c in instancia.component_data_objects(pyo.Constraint, active=True):
 print(f"\t{c} = {c()}")

# Muestra la instancia resuelta
instancia.display()
```

El fichero con los datos del problema de Netflix representado como un PFCM sería `PFCM_1.dat`,[6] cuyo contenido sería:

```
## Ejemplo Netflix como PFCM - Data en formato AMPL

set Nodos:= O S1 S2 S3 C1 C2;

set Arcos:= (O,S1) (O,S2) (O,S3) (C1,O) (C2,O) (S1,C1) (S1,C2) (S2,C1) (S2,C2) (S3,C1) (S3,C2)
    ;

param Costes :=
S1 C1  6
S1 C2  3
S2 C1  2
S2 C2  3
S3 C1  5
S3 C2  4;

param l :=
C1 O 3500
C2 O 4000;

param u :=
O  S1 3000
O  S2 2000
O  S3 2600
C1  O 3500
C2  O 4000;
```

Para resolver el problema de Netflix son el arco desde el Servidor 1 al Cliente 2, el conjunto de datos sería:

```
## Ejemplo Netflix como PFCM - Data en formato AMPL

set Nodos:= O S1 S2 S3 C1 C2;

set Arcos:= (O,S1) (O,S2) (O,S3) (C1,O) (C2,O) (S1,C1) (S2,C1) (S2,C2) (S3,C1) (S3,C2);

param Costes :=
S1 C1  6
S2 C1  2
S2 C2  3
S3 C1  5
S3 C2  4;

param l :=
C1 O 3500
C2 O 4000;

param u :=
O  S1 3000
```

[6] Disponible en la web de la asignatura

```
20 O   S2 2000
21 O   S3 2600
22 C1   O 3500
23 C2   O 4000;
```

4. Ejercicios

1. (**Problema de transporte**) Un empresario dispone de tres almacenes (A_1, A_2 y A_3) y tres puntos de venta (V_1, V_2 y V_3). En cada punto de venta tiene apalabradas 3 toneladas de mercancía.En el almacén A_1 tiene disponibles 4 toneladas de mercancía, 2 toneladas en el almacén A_2 y 3 toneladas en el almacén A_3. Los costes de transporte de cada almacén a cada punto de venta son los dados en la tabla siguiente:

	A_1	A_2	A_3
V_1	3	4	No hay transporte
V_2	1	2	3
V_3	No hay transporte	No hay transporte	3

   ¿Qué cantidad de mercancía se debe transportar desde cada almacén a cada punto de venta para que, satisfaciéndose todos los compromisos, los costes de transporte sean mínimos?

   a) Resolver el problema como un problema de Transporte y realiza un informe de sensibilidad indicando
      - entre qué valores se podrían modificar las disponibilidades de los almacenes y las demandas de los puntos de venta sin que cambie la solución básica factible
      - Como cambiaría el coste de transporte total por cada unidad que se puedan modificar las disponibilidades y las demandas.
   b) Resolver el problema como un problema de flujo en red a coste mínimo.

2. (**Problema de transporte**) Una empresa suministradora de agua dispone de dos depósitos (D_1, D_2) cuyas capacidades son 45 y 65 hectómetros cúbicos. Cada día la empresa debe suministrar el agua a tres pueblos A, B y C, cuyas demandas de agua son 30, 40 y 30 hectómetros cúbicos, respectivamente. El coste del transporte (por hectómetro cúbico de agua) está dado en la tabla siguiente:

	A	B	C
D_1	6	7	4
D_2	5	8	6

   ¿Qué cantidad de agua se debe enviar desde cada depósito a cada ciudad para que, satisfaciéndose todas las demandas, los costes de transporte sean mínimos?

   a) Resolver el problema como un problema de Transporte y realiza un informe de sensibilidad indicando
      - entre qué valores se podrían modificar las disponibilidades de los depósitos y las demandas de las ciudades sin que cambie la solución básica factible
      - Como cambiaría el coste de envío de agua total por cada unidad que se puedan modificar las disponibilidades y las demandas.
   b) Resolver el problema como un problema de flujo en red a coste mínimo.

3. (**Problema de asignación**) Tenemos 9 alumnos que han de realizar prácticas en dos laboratorios. El primero tiene capacidad para dos alumnos y el segundo para tres. Por tanto, hay que distribuir los alumnos en turnos: T_1 y T_2 para el primer laboratorio y T_3 y T_4 para el segundo. Los alumnos han mostrado el siguiente orden de preferencia sobre los turnos (1 prioridad máxima, 4 prioridad mínima):

	T_1	T_2	T_3	T_4
A_1	3	2	1	4
A_2	1	2	3	4
A_3	1	4	3	2
A_4	3	2	4	1
A_5	2	1	3	4
A_6	3	2	1	4
A_7	2	1	3	4
A_8	4	1	2	3
A_9	3	1	2	4

Encontrar la asignación de alumnos a turnos que minimice el descontento total de los mismos (por ejemplo, si el alumno A_1 es asignado al T_1, su descontento será de 2).

a) Resolver el problema como un problema de Transporte.[7]

b) Resolver el problema como un problema de flujo en red a coste mínimo.

[7] Pista: Considerar los turnos como orígenes y los alumnos como destinos